

UAFORGE PLATFORM SPECIFICATION

System Architecture, Datasets, Consensus Engine & API Reference

Production Grade User-Agent Research, Multi-Source Normalization, & Zero-Duplicate Guarantee

1. Executive Project Overview

UAForge is a production-grade User-Agent intelligence, research, validation, and generation engine. Unlike naive generators that assemble arbitrary, synthetic substrings, UAForge operates on an absolute reality mandate: zero synthetic fabrication. Every User-Agent served by the platform originates from real-world telemetry feeds, public data consensuses, and validated browser distributions.

The platform addresses the critical vulnerabilities in web scraping, automated testing, cybersecurity research, and anti-bot mitigation where synthetic or malformed User-Agents cause immediate CAPTCHA challenges, IP blocks, or request drops due to structural and architectural impossibilities.

2. Core Architectural Pillars

- **Zero Synthetic Fabrication:** Eliminates randomized or fictitious version combinations. UAs must trace to concrete browser versions and genuine operating systems.
- **Multi-Source Consensus Engine:** Cross-references strings across 4 distinct data sources: Local High-Confidence Corpus, Microlink HQ, Intoli Real-World Browser Feeds, and WhatIsMyBrowser Commercial API.
- **Zero-Duplicate Guarantee:** Implements private client-isolated exclusion tracking via a dedicated PostgreSQL/SQLite ledger. Once a User-Agent is delivered to a client, it is never served to that client again.
- **Deterministic Confidence Matrix:** Evaluates structural validity, OS-browser compatibility, parser completeness, device consistency, and source corroboration into a normalized 0-100 score.
- **Strict 0-39 Suspicious Quarantine:** Aggressively flags and penalizes incompatible combinations (e.g., modern Safari on Windows) with explicit warning banners and exclusion from priority queues.

3. Source Providers & Data Ingestion Pipeline

UAForge unifies four specialized ingestion providers through an abstracted interface (UserAgentProvider). Ingested items are normalized, deduplicated by exact string hash, parsed for structural tokens, and stored in the primary database.

Provider 1: Local High-Confidence Verified Corpus (Internal DB)

A curated, seed repository of modern stable desktop, tablet, and mobile browsers (Chrome 120-132, Firefox 128-133, Safari 17-18, Edge 120-131) pre-tested against strict platform compatibility standards. Serves as the instant, ultra-low latency fallback baseline with zero network dependencies.

Provider 2: Microlink HQ Top User-Agents (microlinkhq/top-user-agents)

Aggregates high-traffic browser headers actively utilized across the web. The ingest pipeline streams directly from official GitHub raw endpoints (desktop.json, mobile.json, index.json) and Kikobeats crawler agents, backed by jsDelivr CDN fallbacks. It provides high-frequency real-world distribution data.

Provider 3: Intoli Real-World User-Agents (Intoli Dataset)

Derived from real browser visits across thousands of websites. Ingested via live GZIP streaming (user-agents.json.gz). Intoli data contains rich hardware metadata including deviceCategory, platform, viewport dimensions, and visit probability weights, allowing UAForge to prioritize high-frequency genuine web traffic.

Provider 4: WhatIsMyBrowser API (Commercial Parser & Verification)

Leveraged as an authoritative commercial verification layer. Communicates with `api.whatismybrowser.com/api/v2/user_agent_parse`. Validates software recognition, operating system flags, hardware types, and official authenticity consensus.

Automated Background Sync Worker

UAForge runs an automated daily synchronization worker (cron job) and supports on-demand manual triggers via `POST /api/v1/sources/:id/sync`. During synchronization, the pipeline runs a 5-step lifecycle: (1) Fetch raw strings, (2) Structural validation, (3) Parser extraction, (4) Compatibility audit, and (5) Upsert into the relational database with updated consensus counts.

4. Confidence Scoring Engine & Logic Matrix

The confidence score is a deterministic, transparent composite metric (0 to 100) calculated in `ConfidenceService.ts`. It evaluates both empirical corroboration (source presence) and intrinsic syntactic/compatibility characteristics.

Mathematical Weight Distribution

- **Base Single-Source Presence (+48 pts):** Awarded immediately if the User-Agent is discovered in at least one verified real-world source dataset.
- **Multi-Source Corroboration (+20 pts per source, max +40 pts):** Extra points awarded for each independent source that contains the string. Two sources = +20, three+ sources = +40.
- **Platform Compatibility Status (+16 pts / -30 to -65 pts):** Awarded if OS, browser, and hardware tokens align. If an impossible combination is detected (e.g. Safari on Windows), a severe -55 pt penalty is levied.
- **Parser Completeness (+14 pts):** Proportional to the percentage of standard tokens recognized (browser name, version, major version, OS name, OS version).
- **Device Consistency (+8 pts):** Validates that mobile OS (Android, iOS) aligns with mobile device flags, and desktop OS aligns with desktop flags.
- **Recency Bonus (+5 pts):** Awarded if the data source was verified within active synchronization windows.

Confidence Score Tiers

- **90 – 100 (Very High Confidence):** Top-tier production grade. Multi-source verified, flawless platform compatibility, zero anomalies. Prioritized by default in generation APIs.
- **75 – 89 (High Confidence):** Robust real-world browser with full syntactic validity and proven distribution.
- **60 – 74 (Medium Confidence):** Valid User-Agent present in single data source or with minor legacy browser flags.
- **40 – 59 (Low Confidence):** Syntactically valid but rare or aging version distribution.
- **0 – 39 (Suspicious Range):** QUARANTINE ZONE. Contains impossible pairings, missing headers, or structural anomalies. High probability of immediate CAPTCHA or anti-bot challenge. Accompanied by critical visual warning banners.

Deterministic Compatibility Rules (`CompatibilityValidator.ts`)

- **Safari on Windows/Android:** Safari discontinued Windows support after v5.1.7 (2012). Any modern Safari (v6+) on Windows or Android is flagged as INCOMPATIBLE.
- **Internet Explorer on Mobile/Linux:** IE was never released natively for Android, iOS, or modern Linux. Flagged as INCOMPATIBLE.
- **EdgeHTML on Mac/Linux:** Edge versions < 79 were proprietary EdgeHTML (Windows only). Chromium Edge (79+) is valid across all OS.
- **Header Envelope & Parentheses:** Must begin with Mozilla/ or Opera/ and maintain strictly balanced opening and closing parentheses.

5. Zero-Duplicate Delivery Guarantee

A primary pain point in automated workflows and testing is duplicate delivery, which leads to repeated sessions with the exact same fingerprint. UAForge solves this with a cryptographically tracked, client-isolated exclusion ledger.

Database Schema & State Tracking

A dedicated table `served\_user\_agents` records every transaction:

```
CREATE TABLE served_user_agents (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  client_id VARCHAR(100) NOT NULL,  
  user_agent_id INTEGER NOT NULL REFERENCES user_agents(id),  
  served_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(client_id, user_agent_id)  
);
```

Query Execution Flow (Zero Overlap)

• **Client Identification:** Requests transmit a client identifier via query param (?clientId=), header (x-client-id), or authenticated session token.

• **Dynamic SQL Exclusion:** Database queries evaluate: `WHERE id NOT IN (SELECT user\_agent\_id FROM served\_user\_agents WHERE client\_id = ?)` alongside platform, country, and confidence criteria.

• **Atomic Registration:** Upon selection, delivered IDs are immediately inserted into `served\_user\_agents`. Consecutive batch calls guarantee 0% duplicate collision.

• **Audit & Reset APIs:** Clients can view their total delivered count via GET /api/v1/generate/served-stats and purge their exclusion history with POST /api/v1/generate/reset-served.

6. Multi-Platform Responsive Design Standards

The user interface is engineered for seamless operation on both Mobile Phones and Desktop PCs:

• **Desktop Experience:** Expansive 1280px container with live system telemetry, multi-column bento grids, and full data-table views.

• **Mobile Smartphone Experience:** Thumb-accessible fixed bottom navigation bar (fixed bottom-0), smooth sliding hamburger drawer, adaptive card layouts replacing wide tables, and minimum 44px touch targets.

7. Complete REST API Reference

Base Endpoint: ``/api/v1`` (Accepts & returns JSON payloads).

1. Generate User-Agents: POST `/api/v1/generate`

Generates zero-duplicate, real-world User-Agents based on targeting criteria:

```
// Request Body
{
  "quantity": 5, // 1 to 50 (default: 1)
  "minimumConfidence": 80, // 0 to 100 (default: 80)
  "platform": "Windows", // Optional: Windows, macOS, Linux, Android, iOS
  "deviceType": "desktop", // Optional: desktop, mobile, tablet
  "query": "client_abc_123" // Optional: Chrome, Firefox, Safari, Edge
  "country": "US", // Optional: US, GB, DE, JP, etc.
  "data": { "servedCount": 42, "lastServedAt": "2026-09-11T10:20:00Z" },
  "clientId": "client_abc_123" // Private isolation ledger token
}
```

2. Zero-Duplicate Served Stats: GET `/api/v1/generate/served-stats`

Query params: ``clientId=client_abc_123`` // Optional: Chrome, Firefox, Safari, Edge

3. Reset Served Pool: POST `/api/v1/generate/reset-served`

Payload: ``{ "clientId": "client_abc_123" }``. Clears exclusion ledger so all records can be delivered again.

4. Inspect & Analyze User-Agent: POST `/api/v1/analyze`

Evaluates an arbitrary User-Agent string against the full compatibility and confidence rules:

```
// Request Body: { "userAgent": "Mozilla/5.0 ..." }
// Returns: { score: 92, status: "Very High Confidence",
//           compatibility: { isValid: true, checksPassed: [...] },
//           warnings: [], breakdown: { baseSourceScore: 48, ... } }
```

5. System Health & Consensus: GET `/api/v1/health`

Returns system status, database connection, uptime, and latency checks for all 4 providers.

6. Trigger Source Synchronization: POST `/api/v1/sources/:id/sync`

Triggers real-time synchronization for a specific source (``local``, ``microlink``, ``intoli``, ``whatismybrowser``, or ``all``).

7. Generation History & Audit: GET `/api/v1/history`

Returns historic generation batches, parameters used, timestamps, and delivered record counts.